1. Executive Summary

Intellisys is a multi-agent research platform that accepts a natural-language research objective from a user and produces a cited, structured Markdown report. A user submits a request through a browser dashboard; a FastAPI backend validates configuration and hands the request to a LangGraph-orchestrated pipeline of five agents — Supervisor, Research, Analyst, Critic, and Writer — which plan, retrieve, analyze, critique, and write the final output. Every step is persisted to a SQLite database and streamed to the frontend's Live Monitor via polling, giving the user real-time visibility into a multi-minute agentic workflow.

The system's defining design principle is 'no mock mode': every code path that can call an external API (OpenRouter for reasoning, Tavily for search) either succeeds against the real service or fails explicitly with a typed error. No synthetic or fallback data is ever substituted, which keeps failures honest and debuggable rather than silently degraded.

2. System Overview

A user types a research question into the dashboard's New Research form. The request travels to a FastAPI backend, which — after verifying both OpenRouter and Tavily are configured — creates a workflow run record and schedules the actual multi-agent execution as a background task, returning immediately so the UI can switch to a live-updating monitor view.

The background task invokes a compiled LangGraph graph. Five agents execute in sequence, with one conditional loop: the Supervisor plans research questions, the Research agent retrieves evidence for all questions concurrently, the Analyst synthesizes that evidence, the Critic either approves the synthesis or sends it back to the Analyst for revision (bounded by a hard cap), and the Writer produces the final cited report. Every stage transition, evidence item, and log event is written to SQLite as it happens, so a run that fails partway through leaves a full audit trail rather than losing state.

3. High-Level Architecture

```
Your browser (dashboard)  |  HTTP requests (fetch/JSON)  v FastAPI backend  (app/main.py)  |  +-- SQLite database (workflow state, evidence, reports, logs)  |  +-- LangGraph workflow (app/graph/workflow.py)  |  +-- Supervisor Agent  -> OpenRouter (LLM)  +-- Research Agent  -> Tavily (web search)  +-- Analyst Agent  -> OpenRouter (LLM)  +-- Critic Agent  -> OpenRouter (LLM)  +-- Writer Agent  -> OpenRouter (LLM)
```

This document maps that implementation onto the target reference architecture used for Week 4 evaluation. Section 9 (Design Trade-offs) and the companion Known Limitations document detail where the current implementation differs from the reference diagram — principally, there is no explicit clarification-request branch back to the user and no separate parallel Task-A/B/C fan-out label, though research questions are in fact executed concurrently.

4. Technology Stack

Technology	Role
FastAPI	Web server; defines HTTP endpoints, validates request/response shapes, serves frontend static files.

Technology	Role
LangGraph	Orchestrates the 5 agents as a graph of nodes and edges, including the conditional Critic-to-Analyst loop.
Pydantic	Strict schemas for all data moving between agents and through the API.
SQLAlchemy	ORM layer defining and querying the 5 database tables via Python classes.
SQLite	Single-file embedded database (research_platform.db); no separate DB server.
Alembic	Migration scaffolding for future schema changes (currently schema is created via create_all() on startup).
httpx	HTTP client used to call the OpenRouter and Tavily APIs.
tenacity	Retries genuine network failures (timeouts, 5xx) up to 3 times with exponential backoff.
OpenRouter	Single API surface providing access to multiple LLMs; used by all reasoning agents.
Tavily	Web search API purpose-built for AI agents; used exclusively by the Research agent.
Vanilla HTML/CSS/JS	Frontend with no build step; Lucide icons and marked.js loaded from CDN.

4.1 Why Each Technology Was Selected

- FastAPI: async-native, so it can schedule the multi-minute agent workflow as a non-blocking background task while returning an instant HTTP response, and its built-in Pydantic integration keeps request validation consistent with the rest of the schema layer.
- LangGraph: the workflow is not a straight pipeline — the Critic can send work back to the Analyst. LangGraph expresses this as declarative conditional edges instead of hand-rolled if/else control flow, which keeps the graph's shape (and its termination guarantees) easy to reason about and test in isolation.
- Pydantic: every handoff between agents and every API response is schema-validated, so a malformed LLM response is caught immediately as a typed error rather than silently propagating bad data downstream.
- SQLAlchemy + SQLite: the project's scale (a single-user or small-team research tool) does not need a networked database server; SQLite gives zero-ops persistence while SQLAlchemy keeps the data-access code portable if a heavier database is needed later.
- tenacity: isolates retry logic to genuine transient network failures. A previous version of the code wrapped an entire LLM call (including its configuration check) in a blanket retry, which buried a simple 'missing API key' error inside a confusing three-attempt RetryError — the current split design (see Section 8, Backend Architecture) fixed that.
- OpenRouter: gives access to multiple LLM providers behind one API key and one client integration, avoiding a hard dependency on a single model vendor.
- Tavily: purpose-built for agentic retrieval, returning cleaner structured results than scraping a general search engine would.
- Vanilla HTML/CSS/JS frontend: no build step or framework overhead for a dashboard whose interactivity needs (tab switching, polling, Markdown rendering) are well served by plain JS; keeps the deployment footprint to static files served directly by FastAPI.

5. Backend Architecture

The backend is organized around a single FastAPI application (app/main.py) that mounts a REST API router (app/api/routes_workflow.py) and, as its final route, serves the entire frontend directory as static files (with html=True so / resolves to index.html).

5.1 Request Handling

POST /api/workflows is the single entry point for starting a new research run. It performs a fail-fast configuration check (both openrouter_configured and tavily_configured must be true) before creating any database state, returning HTTP 400 with a message naming exactly which key is missing if the check fails. On success it creates a WorkflowRun row with status='queued', schedules the actual graph execution as a FastAPI BackgroundTask, and returns the new run's id immediately — the multi-minute agent pipeline runs after the HTTP response has already been sent.

5.2 Configuration & the 'No Mock Mode' Philosophy

Settings (a Pydantic BaseSettings class) reads OPENROUTER_API_KEY, OPENROUTER_MODEL, TAVILY_API_KEY, DATABASE_URL, and MAX_REVISIONS from the environment. Two computed boolean properties — openrouter_configured and tavily_configured — are checked twice: once at the API boundary (fail fast, before any work starts) and once again inside LLMClient.complete() and web_search() at the point of the actual network call, in case configuration changes mid-run. There is no code path anywhere that substitutes synthetic data for a missing key; both checks raise typed exceptions (LLMNotConfiguredError, SearchNotConfiguredError) instead.

5.3 LLM Client Design

app/llm_client.py wraps OpenRouter's /chat/completions endpoint. complete() checks configuration first and raises immediately (no retry) if missing, before delegating to a separate _call_openrouter() method that is wrapped in tenacity's @retry decorator. This split exists specifically because an earlier version wrapped the whole complete() method in the retry decorator, which meant a simple missing-API-key error got retried three times and surfaced as a confusing RetryError instead of a clear configuration error. parse_json() strips markdown code fences (since some models wrap JSON output in ```json blocks even when told not to) and raises ReportGenerationError on any parse failure, rather than returning a default value.

6. Frontend Architecture

The frontend is a single-page dashboard with no build step: index.html defines the app shell (sidebar, topbar, and one <section class="page"> per tab, toggled via CSS), css/theme.css defines dark/light design tokens as CSS variables swapped via a data-theme attribute, and css/dashboard.css handles layout for widgets, the workflow graph, agent monitor cards, timeline, and report viewer.

js/api.js is a thin fetch() wrapper, one function per backend endpoint. js/dashboard.js contains all interactivity: tab switching, the New Research form, stats widgets, the animated workflow graph, and — critically — pollMonitor(), which calls GET /api/workflows/{id} every 1.5 seconds while the Live Monitor tab is active and re-renders the workflow graph, agent cards, and timeline from whatever state the database currently holds. Polling stops once status becomes completed or failed. The final report is rendered from Markdown to HTML via marked.js, with a clickable table of contents built from the report's own headings.

The initial theme is set by a small inline script at the very top of index.html's <head>, before any CSS loads, specifically to avoid a flash of the wrong theme on page load; js/theme.js then owns reading/writing the preference to localStorage and syncing toggle buttons for the remainder of the session.

7. Database Architecture

Five tables, created automatically on startup via SQLAlchemy's Base.metadata.create_all() (no manual migration step is currently required, though Alembic is present for future schema evolution):

Table	Stores	Written by
workflow_runs	One row per research request: objective, status, current_stage, revision_count, timestamps, error message.	Created on launch; updated by every graph node as it progresses.
tasks	Reserved for per-question task tracking.	Not currently populated (see Known Limitations).
evidence	Every search result: title, URL, snippet, and which question it answers.	Research agent, immediately after each Tavily call.
reports	Final Markdown report(s), versioned.	Writer agent, once at the end of a successful run.
execution_logs	Every event any agent logs, timestamped.	Every agent, via BaseAgent.log_event().

app/database.py configures the SQLAlchemy engine and a SessionLocal factory; FastAPI's dependency injection (Depends(get_db)) hands each API request its own session automatically, and each LangGraph node opens and closes its own session independently.

8. Multi-Agent Architecture

Each agent is a focused, single-responsibility Python class inheriting from BaseAgent, which supplies a shared LLMClient instance and a log_event() method. Tool access is scoped per agent by convention: the Supervisor and Critic call no tools beyond the LLM itself, the Research agent calls only Tavily, the Analyst reads evidence but never fetches new evidence, and the Writer only 'exports' (formats) the final text. Full specifications for each agent — including allowed/prohibited actions, prompt strategy, and failure conditions — are documented separately in Agent Specifications.docx.

9. LangGraph Workflow

app/graph/workflow.py wires the five agents into an executable graph:

```
graph.add_edge("supervisor", "research") graph.add_edge("research", "analyst") graph.add_edge("analyst", "critic") graph.add_conditional_edges( "critic", route_after_critic, # returns "revise" or "approved" {"revise": "analyst", "approved": "writer"}, ) graph.add_edge("writer", END)
```

Each node function updates current_stage on the WorkflowRun row (which the dashboard's animated graph reads), invokes the corresponding agent, and writes results into a shared WorkflowState TypedDict (app/graph/state.py). No node ever receives another node's raw conversation history — only these explicitly typed fields (objective, plan, evidence, analysis, critic feedback, final report, revision_count, status), which keeps context passed between agents minimal and auditable.

The revision loop: if the Critic returns REVISION_REQUIRED, route_after_critic() increments revision_count and routes back to the analyst node — not back to research, since the evidence itself does not change, only its interpretation. This repeats until the Critic approves, or revision_count reaches max_revisions (default 2), at which point critic.py's hard cap forces an APPROVED verdict regardless of what the LLM returned, guaranteeing termination.

10. Component Responsibilities

Component	Responsibility
FastAPI app	HTTP boundary, config validation, background task scheduling, static file serving.
LangGraph workflow	Orchestration and control flow between agents, including the revision loop.
Agents (5)	Domain logic: planning, retrieval, analysis, review, writing.
SQLAlchemy models	Durable state: runs, evidence, reports, logs.
LLMClient	Single point of contact with OpenRouter; retry policy and JSON parsing.
search_tool	Single point of contact with Tavily.
Frontend (dashboard.js)	Polling, rendering, and user interaction; no business logic.

11. Scalability

- Research is the only inherently parallel stage today, implemented via asyncio.gather() over all research questions in a single run — this reduces wall-clock time for a single run but does not, by itself, address multiple concurrent runs.
- Because SQLite is a single file, high concurrent write volume across many simultaneous runs is the most likely first scaling bottleneck; migrating DATABASE_URL to a networked engine (e.g. PostgreSQL) via SQLAlchemy's existing abstraction is the natural next step and requires no application-code rewrite.
- FastAPI's BackgroundTask model runs workflows in-process; a higher-throughput deployment would move to a dedicated task queue (e.g. Celery/RQ) so the API process is not also carrying the agent workload.

12. Reliability

- Evidence is persisted immediately after each Tavily call rather than held in memory, so a failure in the Analyst, Critic, or Writer stage does not lose already-retrieved evidence.
- The revision loop has a hard, code-enforced upper bound (max_revisions), so the workflow cannot hang indefinitely regardless of what the Critic's LLM call returns.
- The entire graph invocation is wrapped in a single outer try/except in the background task; any unhandled exception anywhere in the pipeline is caught once, the run is marked failed, and the real exception message is stored in the error column rather than being lost.

13. Error Handling

Two custom exception types anchor the 'no mock mode' philosophy: LLMNotConfiguredError and SearchNotConfiguredError (both defined in app/exceptions.py), raised the moment a required API key is missing, with no synthetic fallback response. A third failure mode — invalid JSON from an LLM — raises ReportGenerationError from LLMClient.parse_json(), treating a parse failure as a genuine run failure rather than something to paper over with default content. All three exception types, along with genuine network errors, are ultimately caught by the outer try/except in _run_workflow_background(), which marks the WorkflowRun as failed and records the real error message for display in the dashboard.

14. Design Trade-offs

Decision	Trade-off accepted
Polling (1.5s) instead of WebSockets/SSE for Live Monitor	Simpler client and server code, at the cost of slightly delayed updates and more redundant HTTP requests than a push-based channel.
SQLite instead of a networked database	Zero operational overhead for a single-instance deployment, at the cost of limited write concurrency if usage grows.
No mock/fallback mode for LLM or search failures	Failures are honest and debuggable, at the cost of the system being unusable the moment either external API is unreachable or misconfigured — there is no degraded mode.
Revision loop returns to Analyst, not Research, on REVISION_REQUIRED	Faster revision cycles since evidence isn't re-fetched, at the cost of being unable to address a critique that stems from genuinely missing evidence rather than a flawed interpretation of existing evidence.
Writer freely chooses report structure per topic	Produces reports that fit their subject matter rather than a generic template, at the cost of less predictable, harder-to-diff output structure across runs.
References section built deterministically, not by the LLM	Guarantees no hallucinated sources in the References list, at the cost of the LLM being unable to add editorial context (e.g. grouping or annotating sources) there.

15. Deviations from the Reference Architecture

The reference architecture used for Week 4 evaluation includes an explicit 'Clarification Required?' branch back to the user, and depicts three labeled parallel research tasks (Research Task A/B/C) feeding a shared Evidence Store. The current implementation reflects both concepts with the following differences, which are also listed in Known Limitations.docx:

- No clarification loop: the Supervisor always produces a plan from the objective as given; it never pauses execution to ask the user a clarifying question before proceeding.
- Research parallelism is not labeled as fixed 'Task A/B/C' slots — it is a dynamic fan-out over however many questions (3–6) the Supervisor's plan contains, executed concurrently via asyncio.gather() rather than as three named branches.
- The Evidence Store in the reference diagram corresponds directly to the evidence SQL table, which the Analyst reads from — this part of the implementation matches the reference architecture closely.